Forecasting Room Occupancy for HVAC Scheduling

MMA3001 Numerical Methods and Machine Learning - Project Report (Dataset 2: Monash Smart Infrastructure)

Harvey October 2026

Summary. This project uses Monash occupancy sensor data to predict how many people will be in a room over the next hour, so that heating, ventilation and air conditioning (HVAC) only runs when a room is actually going to be used. My final model, a small neural network, predicts the next hour's headcount with an average error of 0.28 people. That is a small but real improvement on simply assuming the next hour will look like the last one. As a second part, I tested whether a physics model of CO_2 build-up could estimate occupancy from a CO_2 sensor instead. It produced sensible ventilation rates, but the CO_2 signal turned out to be too noisy to replace the occupancy sensors.

1. Engineering problem

Context. Ventilation in a building is sized for the number of people in each room, because people breathe out CO_2. Most buildings still run their HVAC on a fixed timetable, such as 7am to 7pm on weekdays. As a result, empty rooms get heated, cooled and ventilated anyway, while busy rooms can still end up stuffy.

Problem. The question I set out to answer was:

> *At the start of each hour, predict the average number of people in each room for the coming hour, using only the data available at that time.*

I also looked at a smaller second question: could a CO_2 sensor, which is cheaper and more common than a people counter, be used to estimate how many people are in a room?

Why it matters. An hour's warning gives the HVAC system time to prepare a room that will be used and to turn down one that won't. The two possible mistakes have different costs. Predicting too many people wastes energy, while predicting too few leaves people in a poorly ventilated room. For that reason I report the direction of the model's errors (its bias), not just their size.

Intended use. The forecast is meant to be one input to a room-by-room HVAC schedule. It is not a safety system, so it should be used with a safety margin at busy times.

Scope limits. The data covers five spaces. Only two of them, G.20 and G.25 in Building 60, could be matched to real rooms by finding their IDs in the location spreadsheet, which is itself labelled "Incorrect Data". I don't know the size, use or timetable of any of the rooms. The CO_2 part relies on a single sensor in room G.38 over about 15 weeks, and treats the room as perfectly mixed.

2. Inputs, outputs and data

Data. The Monash Smart Infrastructure data [2] has two main files. The occupancy file contains 9,091,732 rows from five spaces between November 2023 and April 2026. A new row is only written when the number of people in a room changes, so the data is an irregular stream of events rather than regular readings. The environmental file contains 179,447 messages from five CO_2 sensors, with each reading stored inside a block of JSON text. The sensor list names 77 environmental sensors, but the data contains only five, and only one of those (sensor 869 in room G.38) is on the list.

Data audit. Before modelling anything, I checked the data for problems. Each check in `occupancy/audit.py` reports what it finds rather than quietly fixing it. Four issues stood out:

- **Frozen sensors.** A working CO_2 sensor should almost never send exactly the same message twice. Sensors 777, 227 and 241 sent thousands of messages each, but only one, two and three different ones. Sensor 777 kept resending a reading from 6 May 2023 for months. This only shows up in the time the reading was *taken*, which is stored inside the message, not in the time it *arrived*.
- **Usable CO_2 data.** Sensor 869 in room G.38 worked properly from 29 February to 13 June 2024. Sensor 326 also works, but it isn't on the sensor list and its CO_2 is no higher on weekdays than at weekends, so it doesn't seem to be in a room people use. I left it out.

- **Occupancy rows.** There were four duplicate rows, which I removed. There were no negative headcounts and no rows where the status and headcount disagreed.
- **Outages.** Because rows are only written when something changes, a long silence could mean either “nothing changed” or “the sensor was offline”. I decided that if a room with people in it went silent for more than an hour, or an empty room for more than 24 hours, the rest of that gap should count as missing data. This found 217 outages, about 1,190 hours in total. Section 5 tests whether this choice matters.

Inputs and outputs. The table below summarises what the solution takes in and what it produces.

	What it is	Format and units
Input	Occupancy events	room ID, timestamp with time zone, headcount (whole number of people, 0 or more)
Input	CO_2 readings	sensor ID, measurement time, CO_2 in ppm (must be positive; this data ranged from 403 to 646)
Output	Hourly occupancy	time-weighted average headcount for each room and hour (people, 0 or more)
Output	Forecast	predicted average headcount for the next hour, used for the HVAC schedule (people; negative predictions are set to 0)
Output	CO_2 model	air change rate λ (per hour), CO_2 added per person g (ppm per hour per person), simulated CO_2 (ppm)

Invalid or missing inputs. The code checks its inputs and stops with a clear error message if something is wrong. This covers missing columns, wrong data types, timestamps without a time zone, negative headcounts, events in the wrong order, a time step of zero or less, broken JSON, and missing files. Smaller problems are handled without stopping. If one reading inside a JSON message has failed (sensor 326 often sends an error code instead of a value), only that reading is skipped. Outage time is treated as missing, and any hour that is less than half covered by trusted data is dropped. A room the model hasn’t seen before still gets a forecast, based only on the time and its past headcounts. The inputs and outputs of every function are described in its docstring and in the HTML documentation generated with pdoc.

3. Computational approach

With clean data in hand, the project follows the pipeline in Figure 1. Each stage is a separate script, and the whole thing can be re-run with one command.

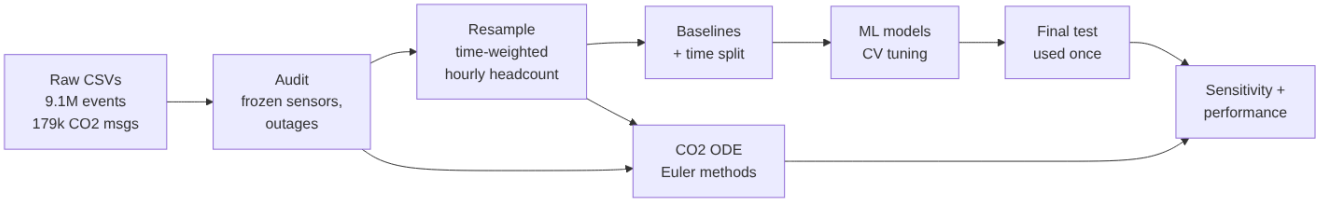


Figure 1. The project pipeline (re-run with `python -m scripts.run_all`).

3.1 Turning events into hourly values

The models need one value per room per hour, but the raw data is a stream of events. Between two events the headcount doesn’t change, so headcount over time is a step function. The correct average for an hour is the area under that step function divided by the time covered (Week 6 numerical integration):

$$\bar{N} = \frac{\int N(t)\, dt}{\int dt} = \frac{\sum_i N_i \, \Delta t_i}{\sum_i \Delta t_i}$$

Because the function is made of flat steps, this sum gives the exact integral. The simpler option, averaging the rows in each hour, gives the wrong answer. Busy periods produce lots of rows because the count keeps changing, while quiet periods produce very few, so busy moments get counted too many times. In a typical hour the row average was 1.4 to 2.1 times too high, and in one hour in ten it was more than 12 times too high. For example, at 9am on 11 April 2024 in G.20, averaging the rows gives 3.6 people, while the true time-weighted average is 0.3. This step produced 84,422 hourly values.

3.2 What the model is allowed to know

Following the "prediction contract" idea from Week 5.6, I fixed exactly what a forecast can use. A forecast made at the start of an hour can see the time of day, the day of the week, whether it's a weekend, which room it is, and the room's **past** headcounts: one, two and three hours ago, the same hour yesterday, the same hour last week, and the average over the last day. The hour and day are converted with sine and cosine so that 11pm sits next to midnight, the same way positions do on a clock face. Past values are taken from a complete hourly timeline, so if an hour is missing because of an outage, its value stays missing instead of being borrowed from the wrong hour. This left 82,215 usable rows.

3.3 Models

I compared five machine learning models from the unit against three simple baselines. The baselines are always predicting the average, persistence ("the next hour will look like the last one"), and the usual average for that room, weekday and hour. The five models, all built with scikit-learn [3], are ridge regression (Week 5.1), a decision tree (5.2), k-nearest neighbours (3.6), a support vector regressor or SVR (5.3), and a neural network with one hidden layer of 32 neurons (5.4). The SVR was trained on a 5,000-row sample, because it becomes very slow on large datasets. Any scaling of the inputs is done inside a scikit-learn `Pipeline`, so it is learned from the training data only. I chose each model's settings, such as tree depth or network size, using `TimeSeriesSplit` cross-validation on the training period.

3.4 CO_2 mass-balance model

For the second question I used a simple ODE from Week 7. If the room is treated as one well-mixed volume of air, its CO_2 rises because people breathe it out and falls because ventilation replaces the air:

$$V\frac{dC}{dt} = G\,N(t) - Q\,(C - C_b) \quad \Rightarrow \quad \frac{dy}{dt} = g\,N(t) - \lambda\,y$$

Here $y = C - C_b$ is the CO_2 above the background level, $g = G/V$ is the CO_2 added per person and $\lambda = Q/V$ is the air change rate. The model assumes the air is perfectly mixed, the background is constant at 429 ppm, everyone breathes out the same amount of CO_2, and the number of people stays the same within each 10-minute step. Only g and λ can be worked out from CO_2 data, because the room volume V, airflow Q and CO_2 per person G only appear as ratios.

I solved the equation with forward Euler, $y_{k+1} = (1 - h\lambda)y_k + hgN_k$, and with backward Euler, $y_{k+1} = (y_k + hgN_k)/(1 + h\lambda)$. To find λ, I looked at 75 nights when the room was empty and CO_2 was falling, and fitted a straight line through $\ln(C - C_b)$ using least squares. Only readings at least 15 ppm above background were used, which also avoids taking the log of readings below it. I then fitted the daytime λ and g using weekdays in March and April.

One problem is that none of the occupancy spaces could be matched to room G.38, so I had to use another room's headcount as a stand-in. I compared all five rooms against the CO_2 after removing the usual weekly pattern from both. Space 3e5a matched best (correlation 0.61), but it has no data from March or April, when the model was fitted. The next best was space 941f (0.50), so I used that. This is the weakest assumption in the project, and I return to it in Section 4.

4. Validation method and results

What was validated. I checked three things: whether the forecast beats the baselines, whether the numerical methods are working correctly (verification), and whether the CO_2 model matches real measurements (validation).

Why this method. I split the data by time. The models were trained on November 2023 to June 2025, compared on July to October 2025, and tested once on November 2025 to April 2026. I avoided a random split because occupancy in one hour is very similar to the hour before (on the validation data, persistence alone has an R^2 of 0.42). With a random

split, the model would see the hours on either side of each test hour, and the results would look better than they really are. A time split tests the model the way it would actually be used: predicting the future from the past.

Metrics and acceptance criteria. My main measure is the mean absolute error (MAE), the average size of the error in people, because it is easy to interpret. I also report the root mean square error (RMSE), which punishes big misses more, and the bias. To be accepted, a model had to beat persistence on the validation data, with a 95% bootstrap confidence interval (Week 5.5) that didn't include zero. I resampled whole days rather than single hours, because errors on the same day tend to go together. All five models passed. The neural network, for example, beat persistence by 0.027 people, with a confidence interval of 0.016 to 0.039.

Did validation influence model selection? Yes. Cross-validation chose each model's settings, and the validation period chose between the models. The SVR, decision tree and neural network finished within 0.003 people of each other, which is inside the confidence intervals, so I broke the tie using RMSE. This picked the neural network (Table 2). I saved this decision in git **before** using the test data, so that the test result couldn't influence the choice.

Table 1. Test results (November 2025 to April 2026). The last column shows the difference in MAE from persistence, with its 95% confidence interval. Negative means better than persistence.

Model	MAE (people)	RMSE	Bias	R^2	MAE, weekdays 8am-6pm	MAE vs persistence (95% CI)
Neural network (selected)	0.283	**0.828**	+0.016	**0.510**	0.552	-0.015 (-0.023 to -0.006)
SVR	**0.275**	0.861	-0.040	0.470	**0.537**	-0.023 (-0.034 to -0.014)
Decision tree	0.282	0.846	+0.016	0.488	0.566	-0.016 (-0.025 to -0.008)
k-nearest neighbours	0.295	0.831	-0.002	0.506	0.573	-0.003 (-0.014 to +0.008)
Persistence	0.298	0.977	0.000	0.317	0.591	-
Ridge regression	0.305	0.857	+0.034	0.474	0.567	+0.007 (-0.003 to +0.017)
Seasonal profile	0.506	1.050	+0.177	0.212	0.944	+0.208
Global mean	0.747	1.202	+0.217	-0.034	0.917	+0.449

Results. On the test data the neural network still beat persistence, by 5% on MAE and 15% on RMSE. The improvement is real but modest. The test results didn't exactly match validation. The SVR ended up with the lowest MAE, but it had a higher RMSE and tended to under-predict, which is the riskier mistake for ventilation. The k-nearest neighbours and ridge models no longer beat persistence at all. Because the neural network had already been chosen and recorded, none of this could change the decision, which is exactly the reason for only using the test data once.

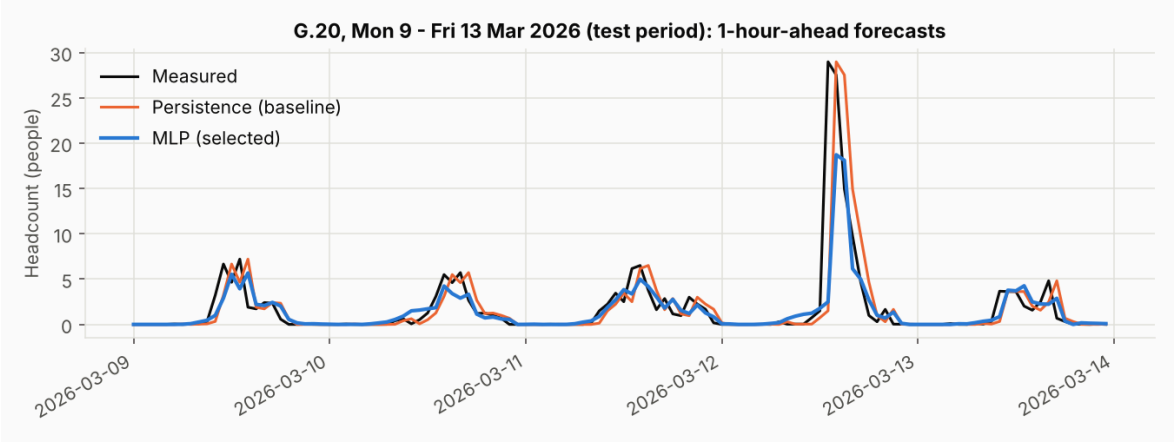


Figure 2. A test week in room G.20, showing the measured headcount and the persistence and neural network forecasts.

Figure 2 shows what the forecasts look like in practice. Both models are about an hour late when people arrive, because they mostly learn from the previous hour. Neither predicted the 29-person spike on 12 March. The same hour the week before had only about four people, so this was a one-off event rather than a regular class, and nothing in the inputs could have signalled it.

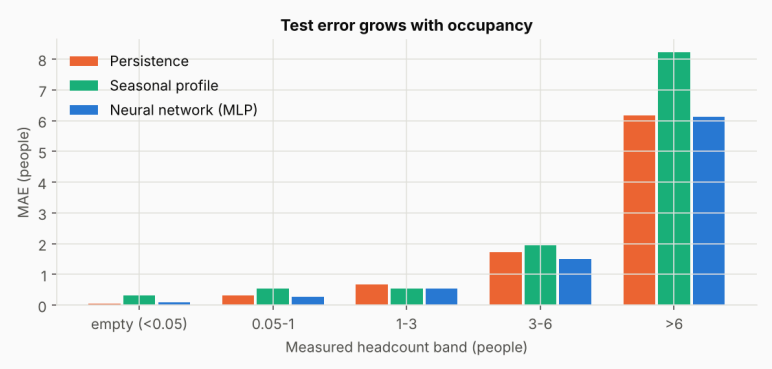


Figure 3. Test error grouped by how many people were actually in the room.

Figure 3 shows that the errors grow as rooms get busier, and that the neural network tends to **under-predict busy hours**: by about 1 person when 3 to 6 people were present, and by about 5 people when there were more than 6. This is its main weakness, because it would lead to under-ventilating rooms at their busiest.

What this means for HVAC. On weekdays between 7am and 7pm, 2,285 of the 5,792 test room-hours (39%) were empty, yet a fixed timetable conditions all of them. Suppose HVAC is turned down whenever the forecast is below half a person. Persistence would turn down 1,869 of the empty hours, but would also wrongly turn down 17% of the occupied hours. The neural network would turn down 1,503 empty hours while wrongly turning down only 6% of occupied ones. It saves slightly less energy but leaves far fewer people in an unventilated room. Because the forecast is late on arrivals, most of this benefit comes from turning HVAC down in hours that stay empty, such as unused rooms, lunch breaks and after classes, rather than from preparing a room before the first person arrives. I couldn't convert these hours into energy savings, because room sizes and HVAC details aren't in the data.

Verification of the numerical methods. Before trusting any results, I checked that each numerical method was doing what it should:

- **Resampling.** The total number of person-hours was the same before and after splitting the data into hours, to within 1.2×10^{-16}, which is the limit of computer precision. A hand-worked example (an empty room for 50 minutes, then 3 people for 10 minutes, which should average 0.5) also came out exactly right.
- **ODE solver.** Both Euler methods matched the exact solution of the ODE for a constant number of people, $y(t) = y_{ss} + (y_0 - y_{ss})e^{-\lambda t}$. Halving the time step halved the error, as expected for a first-order method (Figure 4, left). When $h\lambda = 2.5$, forward Euler blew up while backward Euler stayed stable, just as the theory predicts (Figure 4, right).
- **Parameter fitting.** I generated synthetic CO_2 data from known values of g and λ, and checked that the fitting code gave back exactly those values (a manufactured solution).

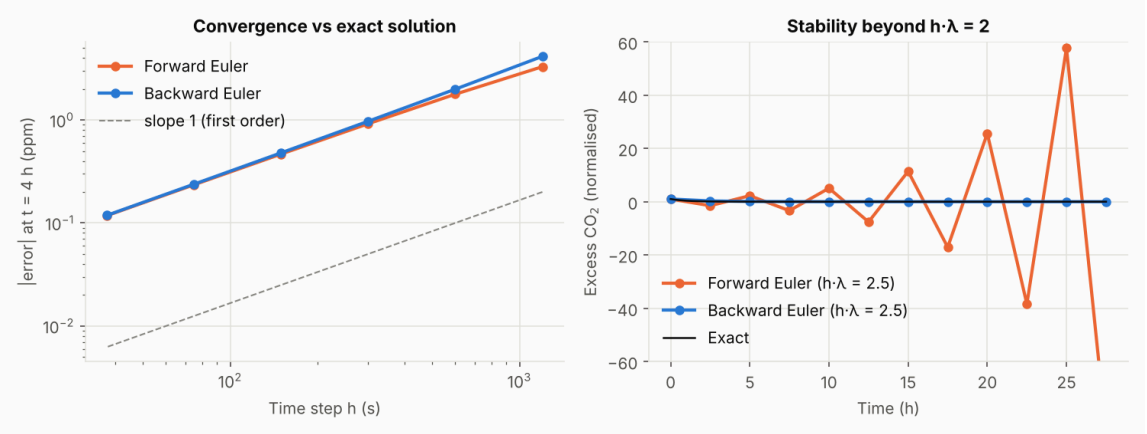


Figure 4. Left: error against time step for both Euler methods. Right: forward Euler becomes unstable when hλ > 2, while backward Euler does not.

Validation of the CO_2 model. The fitted daytime ventilation rate, λ = 0.95 per hour, means the air in the room is replaced about once an hour, which is normal for a mechanically ventilated room. The overnight value of 0.04 per hour is lower than typical air leakage, so the room may be well sealed at night or the background level may drift, and I treat it with caution. The fitted g of 41 ppm per hour per person implies a room of about 450 m³, using the published figure of 0.0052 litres of CO_2 per second for a seated adult [6]. That is plausible, but g also absorbs any size difference between G.38 and the stand-in room, so it is only a rough check. With 10-minute steps, hλ = 0.16, well under the stability limit of 2, so even forward Euler is stable here.

On 31 days that weren't used for fitting, the model's CO_2 error was 38 ppm, compared with 82 ppm if people are ignored (Figure 5). That is a weak comparison, so it mainly shows that occupancy explains a large part of the change in CO_2.

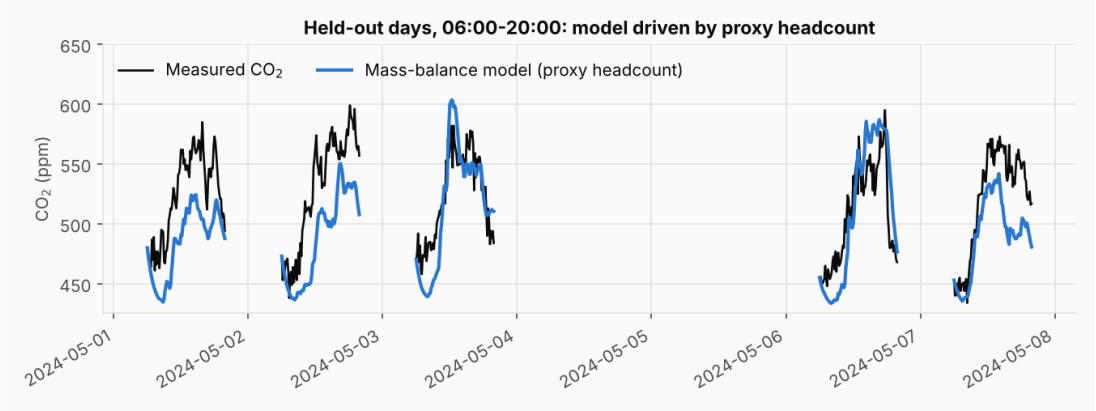


Figure 5. Held-out days: measured CO_2 compared with the model driven by the stand-in room's headcount.

The model drops too low before 8am because it assumes the daytime ventilation rate all day, when the HVAC actually switches on in the morning. This is a problem with the model's assumptions, not with the solver. I also tried working backwards from CO_2 to headcount using central differences (Week 7.1). The result followed the stand-in room's headcount (correlation 0.63), but with an average error of 0.67 people, because taking derivatives of noisy data amplifies the noise.

What validation cannot establish. These results don't show how the model would perform in other buildings or seasons. They can't show whether the occupancy sensors themselves count correctly, because there is nothing independent to compare them against. And they can't show G.38's true occupancy, because the CO_2 test relies on another room's headcount.

5. Comparison and optimisation

5.1 Alternative solutions

Table 2 compares all of the approaches on the validation data, which is the data the choice was based on.

Table 2. Comparison on the validation period.

Approach	Val MAE	Val RMSE	CV MAE (mean ± std)	Train time	Predict time	Interpretable?
Neural network	0.337	**0.836**	0.371 ± 0.074	4.0 s	0.02 s	Low
Decision tree	0.336	0.877	0.374 ± 0.084	0.28 s	0.01 s	High
SVR (5,000 rows)	**0.334**	0.958	0.391 ± 0.114	0.42 s	1.1 s	Low
k-nearest neighbours	0.343	0.847	0.398 ± 0.090	0.04 s	1.1 s	Medium
Ridge regression	0.350	0.878	0.381 ± 0.077	0.05 s	0.01 s	High
Persistence	0.364	0.992	-	-	instant	High

The variation between cross-validation folds is larger than the differences between models, so in practice they all perform fairly similarly. The SVR and k-nearest neighbours are slow at making predictions, because they compare every new point with the stored training data. A Gaussian process would have needed about 23 GB of memory for 54,000 training rows, so I didn't use one. I chose the neural network because it had the lowest RMSE and makes predictions quickly. The decision tree would be a good alternative if being able to explain the model mattered more.

5.2 Sensitivity analysis

The outage limits from Section 2 were my own judgment, so I tested whether changing them would change the results (Table 3).

Table 3. Sensitivity to the outage limits (validation data).

Outage limit (occupied / empty)	Person-hours	Val MAE persistence	Val MAE neural net
0.5 h / 24 h	58,830	0.364	0.341
1 h / 24 h (used)	59,040	0.364	0.337
4 h / 24 h	59,174	0.364	0.335
1 h / 12 h	59,027	0.376	0.350

Person-hours is the total number of people multiplied by hours across all rooms. Each setting slightly changes which hours are included, but the limit for occupied rooms makes almost no difference, so the results don't depend on that choice. Making the empty-room limit stricter (12 hours) drops 1,470 hours that were almost all empty (60% at weekends), which removes about 4,000 usable rows, so I kept 24 hours. For the CO_2 model, I varied the background level between 422 and 445 ppm. This changed g by at most 7%, but the daytime λ by 10 to 16%, so λ is the most sensitive parameter in the physics model.

5.3 Performance analysis

Machine limits. Following the method from Week 2.1 [1], the theoretical peak speed of the computer is F_T = 2 cores × 2.1 GHz × 32 FLOPs per cycle = 134 GFLOP/s. I couldn't install likwid, so I measured the real limits myself. A large matrix multiplication reached 65 GFLOP/s, and a triad test (A = B × C + D) on arrays too big for the cache reached a memory bandwidth of 15.5 GB/s. Dividing one by the other gives a machine balance of 4.2 FLOPs per byte. Timings on a shared machine vary, so every time below is the average of 7 repeats with `timeit`, with its standard deviation.

Table 4. Arithmetic intensity and timing of the main calculations.

Code	FLOPs / bytes per step	Arithmetic intensity	Time	% of roofline
Backward Euler, Python loop (10^6 steps)	4 / 16	0.25	244 ± 0.6 ms	0.4%
Backward Euler, `lfilter` (10^6 steps)	3 / 16	0.19	7.0 ± 1.0 ms	15%
Neural network prediction (117k rows*)	1,120 / 136	8.2	70 ± 0.9 ms	2.9%

*The validation set repeated 10 times, to give a longer and more stable timing.

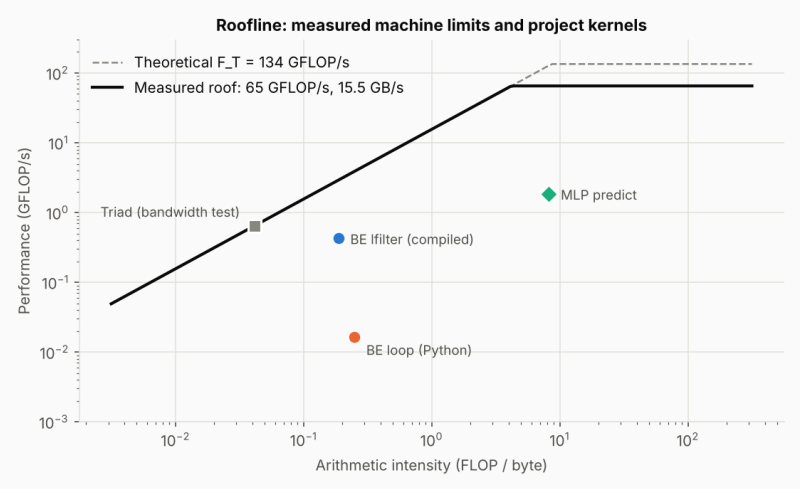


Figure 6. Roofline plot [5] with the measured machine limits and the project's code.

Baseline vs improved. The backward Euler solver does very little arithmetic for the amount of data it moves, so on the roofline (Figure 6) it sits in the memory-limited region. However, the original Python loop only reached 0.4% of that limit. Using line_profiler, I found that all of the time was spent inside the loop itself, about 0.24 microseconds per step, so the bottleneck was Python's overhead rather than the hardware. Backward Euler for this equation is simply "new value = a × old value + b × input", and SciPy's `lfilter` function [4] runs exactly that loop in compiled C. It gives identical answers, agreeing with the Python version to 10^{-12}, and is **35 to 38 times faster** for 10,000 steps or more (about 17 times at 1,000 steps), while still scaling linearly with the number of steps. It still only reaches 15% of the bandwidth limit, because each step has to wait for the one before it, so the real limit is how quickly one step can follow another rather than memory. On this 15-week dataset the loop only takes milliseconds either way, so the speed-up matters most if the model is fitted many times or scaled up to many sensors over several years. The neural network does much more arithmetic per byte (8.2), which puts it in the compute-limited region, but its small layers mean it only reaches 2.9% of peak.

Precision. Switching from float64 to float32 changed the solver's results by only 4×10^{-7}, far less than the 8 ppm sensor noise. It wasn't any faster, though, because each step still has to wait for the last one, so I kept float64.

6. Limitations and lessons learned

Limitations. The forecast under-predicts busy hours, so it would need a safety margin in practice. It is about an hour late when people arrive, so it is more useful for turning HVAC down in empty hours than for preparing a room before people get there. It has no timetable information, so it can't anticipate one-off events. The data only covered five rooms, two of them identified, all from one sensor device, and I had to assume the occupancy sensors count correctly. The CO_2 model assumes perfect mixing and a single daytime ventilation rate, and it relies on another room's headcount.

Lessons learned.

1. **Check the data first.** Most of the listed sensors were missing or broken, and the frozen ones only showed up in the measurement times.

2. **How you average matters.** Averaging rows instead of averaging over time overestimated occupancy by 1.4 to 2.1 times in a typical hour, and by more than 12 times in one hour in ten.
3. **Simple baselines are hard to beat.** Persistence already captured most of the pattern, and machine learning only improved on it by 5 to 15%. It is better to report that honestly than to oversell the model.
4. **Protect the test set.** The SVR happened to do slightly better on test MAE, but because the model was chosen before testing, the test result is an honest check rather than a second chance to pick.
5. **Verification and validation are different.** The solver was verified exactly, but the model was still wrong before 8am because of one of its assumptions.
6. **Profile before optimising.** The solver looked memory-limited on paper, but profiling showed that Python's overhead was the real problem.

Improvements. The most useful next steps would be to add the university timetable as an input, to smooth the CO_2 data before working backwards to headcount, to train the model to lean towards over-predicting (since under-ventilating is the worse mistake), and to put a CO_2 sensor in the same room as an occupancy sensor so the physics model can be tested properly.

7. AI use and reflection

Tools used. Claude (Anthropic, Claude Opus 5.5).

What it was used for and level of contribution.

Task	AI contribution	What I did
Reading the brief and data; suggesting a dataset	Medium	Questioned how it knew the sensors were broken, checked the raw data myself, then chose Dataset 2
Writing the code	High	Set the requirements, reviewed outputs, ran the pipeline
Choosing methods	Low	Chose the 1-hour forecast and scope, and kept it within the unit's content
Tests, docs, performance	High	Checked tests use known answers and reviewed every figure
Report	Medium (drafting)	Directed the structure, tone and length, and reworded the first draft

Why I used it. I mainly wanted help with the coding and testing, which is where I used it most. Analysing a dataset of more than 9 million rows and building and testing several models by myself would have taken a very long time. Using AI for that let me spend more of my time analysing the results and making the report better.

How I checked it. Tests with known answers (the 0.5-person example, the exact ODE solution, manufactured data), looking at raw data and every figure, and comparing claims with the source and git files.

Errors it made.

- It called sensor 326 an "outdoor" sensor, but the sensor isn't on the list, so its location is unknown.
- Its first JSON reader threw away whole messages when one reading had an error, discarding 29,043 messages and the valid CO_2 reading in each.
- Its resampling code used the wrong time units (putting the data in 1970) and stretched each sensor's last reading 24 hours into the future. Unit tests caught both.
- It used a database tool (DuckDB) that isn't taught in the unit. In my scope review it was replaced with pandas, with identical results, and a number it had quoted from early exploration was corrected (0.53 to 0.50).

Decisions that stayed mine. The dataset, the forecast horizon and scope, keeping everything within the unit, the model choice, and what to claim in the report.

Effect on my understanding. Having each step explained helped me follow the reasoning, for example why a random split leaks information or why forward Euler needs $h\lambda < 2$. The main risk was trusting it too much: several of its

mistakes gave no error message and were only caught by tests or a number that looked wrong.

8. References

1. Monash University (2026). *MMA3001 Numerical Methods and Machine Learning*, unit notes Weeks 1.1-11.5.
2. Monash Smart Infrastructure (2026). *Dataset 2: Occupancy and Environmental Sensor Data*, supplied for MMA3001.
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
4. Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272.
5. Williams, S., Waterman, A. and Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65-76.
6. Persily, A. and de Jonge, L. (2017). Carbon dioxide generation rates for building occupants. *Indoor Air*, 27(5), 868-879.
7. Anthropic (2026). Claude Opus 5.5, accessed through claude.ai.

GitHub repository: https://github.com/harveystewart5/mma3001-occupancy-forecasting (includes the HTML code documentation in `docs/api` and the test report in `docs/test_report.html`).